Тема 1. Структура на C++ програма. Основни типове данни – константи, променливи и функции. Операции и изрази. Стандартни библиотечни функции.

Цел: Съставяне, компилиране, тестване и анализ на резултатите от изпълнението на C++ програми с дефиниране и обработка на основни типове данни и използване на стандартни библиотечни функции.

Основни сведения за C++ програмите

1. Езикът за програмиране от високо ниво C++ е удобен и ефективен за проектиране, разработване и поддържане на различни по сложност приложни и системни програми. Той предоставя възможности както за процедурно, така и за обектно-ориентирано и обобщено програмиране.

Предоставя удобства и възможности за използване на множество стандартни функции и класове чрез включване (`#include`) в програмата на съответните заглавни файлове (headers) от стандартната библиотека.

Елементите на програмния език C++ могат да бъдат представени като „йерархична система“, базирана на множество от допустими символи – букви, цифри и специални символи, от които по определени правила се изграждат думите на езика – ключови думи и потребителски идентификатори. От тях, по определени синтактични правила, се изграждат оператори (statements). Като последователност от оператори, в съответствие с определен алгоритъм, се съставят програми, съдържащи главна функция `main()` и при необходимост допълнителни потребителски функции.

2. Общият вид на структурата и съдържанието на C++ програма е:

```
#include <заглавен_файл>

// Деклариране на потребителски функции - прототипи

// Дефиниране на глобални константи/променливи

int main()
{
    // Дефиниране на локални променливи/константи
    // Изпълними оператори

    return 0;
}

// Дефиниции на потребителски функции
```

Общият вид на дефиниция на функция е:

```
тип име(параметри)         // Заглавие на функцията
{
    // Дефиниране на локални променливи/константи
    // Изпълними оператори

    return резултат;       // При функция, връщаща стойност
}
```

Типът определя вида на резултата, който функцията връща чрез оператор от вида:

```
return израз;
```

Резултатът може например да бъде целочислен (`int`, `long long`), символен (`char`), реален (`float`, `double`), логически (`bool`) и др.

При функция, която не връща стойност, се използва типът `void`.

Чрез параметрите се извършва обмен на данни между функциите в програмата.

Пример: дефиниция на функция за пресмятане лицето на квадрат със страна `x` – цяло число, чиято стойност се предава като параметър:

```
int lice_kv(int x)         // Функция за лице на квадрат
{
    int rez;               // Дефиниране на резултата
    rez = x * x;           // Пресмятане
    return rez;            // Връщане стойността на резултата
}
```

Функцията може да бъде записана и по-кратко:

```
int lice_kv(int x)
{
    return x * x;
}
```

3. Данните в C++ програма са константи и променливи.

Променливите могат да получават различни стойности по време на изпълнение на програмата и задължително се представят чрез име – идентификатор. Идентификаторът може да съдържа букви, цифри и символ _, като не може да започва с цифра.

Текущата стойност на променливата се съхранява в съответна област от паметта.

Променливите могат да получават стойности чрез оператор за присвояване, чрез въвеждане от стандартния вход или по друг начин.

Променливата трябва да бъде декларирана преди нейното използване:

```
тип име_на_променлива;
```

или с начална стойност:

```
тип име_на_променлива = начална_стойност;
```

В съвременния C++ се препоръчва променливите да бъдат инициализирани още при създаването им, например:

```
int broi = 0;
double suma = 0.0;
```

Константите имат стойност, която не може да бъде променяна след инициализирането им:

```
const double PI = 3.141592653589793;
```

Когато стойността може да бъде определена още при компилация, може да се използва и:

```
constexpr double PI = 3.141592653589793;
```

За логическо описание на данните, подлежащи на програмна обработка, се използва понятието **тип на данните**.

Типът на данните определя допустимите стойности, операциите, които могат да се извършват с тях, и начина им на представяне в паметта.

Основните вградени типове данни, използвани в C++, са:

- целочислени – `short`, `int`, `long`, `long long`;
- реални – `float`, `double`, `long double`;
- символен – `char`;
- логически – `bool`, със стойности `true` и `false`;
- тип без стойност – `void`.

Освен тях стандартната библиотека предоставя множество допълнителни типове. Например за символни низове често се използва:

```
std::string
```

4. Изрази.

Изразите са програмни конструкции за изчисляване или определяне на стойност – числова, символна, логическа или от друг тип.

Израз може да бъде константа, променлива, извикване на функция или съвкупност от тях, свързани чрез оператори.

Примери:

```
a + b
x * x
```

```
a > b
x >= 0 && x <= 100
std::sqrt(x)
```

Част от израз, заградена в скоби, се нарича подизраз.

5. Оператор за присвояване.

Предназначен е да присвоява стойност на променлива.

Общият му вид е:

```
променлива = израз;
```

Например:

```
suma = a + b;
lice = a * b;
```

Преди присвояването се пресмята стойността на израза и при необходимост тя се преобразува към типа на променливата.

Примерните варианти на описаните по-горе елементи на езика C++ са представени и анализирани в следващите програмни реализации.

Обхванати са алгоритми с използване на програмни конструкции за реализация на структурите:

- последователност;
- избор (разклонение);
- цикъл.

Примерни задачи и програми

Задача 1.1.

Условието на задачата е дефинирано в заглавния коментар на представения по-долу вариант на програмата.

Използвани са стандартните потоци `std::cin` за въвеждане от клавиатурата и `std::cout` за извеждане на екрана, дефинирани в заглавния файл `<iostream>`.

Обработват се целочислени данни чрез операциите деление `/` и остатък от целочислено деление `%`.

За определяне на часа, минутите и секундите се използват оператори за присвояване.

```
/* Лабораторно упражнение 1. Задача 1. */
/* C++ програма, която по въведено от клавиатурата време
   в секунди определя и извежда точното време,
```

```
   изтекло от 00:00:00 часа на текущото денонощие. */

#include <iostream>

int main()
{
    long long tsek;       // Времето в секунди
    int chas, min, sek;   // Час, минута, секунда

    std::cout << "Въведете време в секунди (0 - 86400): ";
    std::cin >> tsek;

    chas = static_cast<int>(tsek / 3600);
    min = static_cast<int>((tsek / 60) % 60);
    sek = static_cast<int>(tsek % 60);

    std::cout << "Часът е "
              << chas << ":"
              << min << ":"
              << sek << '\n';

    return 0;
}
```

Въведете, компилирайте и изпълнете програмата.

Променливата `tsek` е дефинирана като `long long`, което позволява обработването и на значително по-големи целочислени стойности.

За въвеждането и извеждането в стандартен C++ се използват операторите:

`>>`

и

`<<`

съответно с потоците `std::cin` и `std::cout`.

Изпълнете програмата с няколко различни стойности за времето в секунди, например:

```
3600
3690
55555
86400
```

Анализирайте получените резултати.

Възможни ли са други варианти за определяне на точното време?

Както се забелязва, в програмата се изпълнява последователност от оператори без проверка на условия, т.е. алгоритъмът е линеен.

Опитайте да „усложните“ програмата, като позволите въвеждането и на стойности, по-големи от 86400 секунди, и пресметнете броя на изминалите денонощия.

Например:

```
long long dni = tsek / 86400;
chas = (tsek / 3600) % 24;
```

Задача 1.2.

Условието и структурата на примерната програма са подобни на тези в предходната задача – реализация на линеен алгоритъм с използване на структура **последователност**.

Обработват се реални числа чрез стандартните математически функции `std::sqrt()` – квадратен корен, и `std::pow()` – повдигане на степен, дефинирани в заглавния файл `<cmath>`.

За форматирано извеждане на резултатите се използват средствата от заглавния файл `<iomanip>`.

```
/* Лабораторно упражнение 1. Задача 2. */
/* C++ програма за изчисляване и извеждане на екрана
   на периметъра P и лицето S на равнобедрен триъгълник
   по въведени от клавиатурата основа B и височина H
   - реални числа в сантиметри. */

#include <iostream>
#include <cmath>
#include <iomanip>

int main()
{
    double b, h;          // Основа и височина
    double p, s;          // Периметър и лице
    double strana;        // Бедро на триъгълника

    std::cout << "Въведете две числа за B и H на триъгълника: ";
    std::cin >> b >> h;

    strana = std::sqrt(std::pow(b / 2.0, 2) + std::pow(h, 2));

    p = b + 2 * strana;
    s = b * h / 2.0;

    std::cout << std::fixed << std::setprecision(2);

    std::cout << "Основата е " << b
              << " см., а височината е "
              << h << " см.\n";

    std::cout << "Периметърът е "
              << p << " см.\n";

    std::cout << "Лицето на триъгълника е "
              << s << " кв. см.\n";

    return 0;
}
```

Въведете и компилирайте програмата.

Изпълнете я с няколко варианта на входните данни, например за триъгълник с:

```
основа B = 6 см
височина H = 4 см
```

Проверете верността на получените резултати.

Обяснете предназначението на:

```
std::fixed
std::setprecision(2)
```

Опитайте да замените израза:

```
std::pow(b / 2.0, 2)
```

с:

```
(b / 2.0) * (b / 2.0)
```

и сравнете резултатите.

Задача 1.3.

C++ програма за определяне високосна ли е въведена от клавиатурата година.

Годината е високосна, ако:

- се дели на 4 и не се дели на 100;

или

- се дели на 400.

Необходимо е използването на логически израз, съдържащ операциите логическо „И“ (`&&`) и логическо „ИЛИ“ (`||`).

За съхраняване на резултата от логическия израз естествено може да бъде използвана променлива от тип `bool`.

```
/* Лабораторно упражнение 1. Задача 3. */
/* C++ програма за извеждане на екрана
   "Годината ... е високосна.",
   ако въведената година е високосна, или
   "Годината ... не е високосна."
   – в обратния случай. */

#include <iostream>
```

```
int main()
{
    int god;
    bool visgod;

    std::cout << "Въведете цяло число за година: ";
    std::cin >> god;

    visgod =
        ((god % 4 == 0) && (god % 100 != 0))
        || (god % 400 == 0);

    if (visgod)
        std::cout << "Годината " << god << " е високосна.\n";
    else
        std::cout << "Годината " << god << " не е високосна.\n";

    return 0;
}
```

Въведете, компилирайте и изпълнете програмата.

Изпълнете програмата с няколко варианта на въвежданата година, например:

```
2000
2007
2008
2010
3000
1900
1600
1381
```

Проверете верността на получаваните резултати.

Опитайте да съставите по-кратък вариант на програмата без използването на променливата `visgod`.

Например условието може да бъде записано директно в оператора `if`.

Задача 1.4.

Условието и структурата на примерната програма са свързани с обработка на данни от тип `char`.

Програмно се представят ASCII кодовете на въведена малка латинска буква, предходната буква, следващата буква и съответната главна буква.

Забележка: ASCII стандартът включва латинската азбука, цифри и специални символи. Кирилските букви не принадлежат към ASCII и в съвременните системи обикновено се представят чрез Unicode кодировки, например UTF-8. Поради това настоящата задача се изпълнява с латински букви.

За организацията на програмната конструкция цикъл се използва оператор за цикъл с постусловие:

```
do ... while
```

Операторите в тялото на цикъла се изпълняват многократно до въвеждане на `N` или `n`.

```
/* Лабораторно упражнение 1. Задача 4. */
/* C++ програма за въвеждане на малка латинска буква
   и извеждане на ASCII кода на въведената,
   предходната, следващата и съответната главна буква. */

/* Вариант с цикъл do ... while и
   запитване за продължение. */

#include <iostream>

int main()
{
    char mbukva, gbukva;
    char otg;

    do
    {
        std::cout << "\nВъведете малка латинска буква: ";
        std::cin >> mbukva;

        std::cout << "ASCII кодът на въведената буква "
                  << mbukva << " е "
                  << static_cast<int>(mbukva) << '\n';

        std::cout << "Предходната буква е "
                  << static_cast<char>(mbukva - 1)
                  << ", ASCII код "
                  << static_cast<int>(mbukva - 1) << '\n';

        std::cout << "Следващата буква е "
                  << static_cast<char>(mbukva + 1)
                  << ", ASCII код "
                  << static_cast<int>(mbukva + 1) << '\n';

        gbukva = mbukva - ('a' - 'A');

        std::cout << "Съответната главна буква е "
                  << gbukva << '\n';

        std::cout << "ASCII кодът на буквата "
                  << gbukva << " е "
                  << static_cast<int>(gbukva) << '\n';

        std::cout << "\nЖелаете ли да продължите? Y/N: ";
        std::cin >> otg;

    } while (otg != 'N' && otg != 'n');

    return 0;
}
```

Въведете, компилирайте и изпълнете програмата с няколко варианта на входни данни – избрана от вас малка латинска буква.

Проверете верността на получаваните резултати.

Модифицирайте програмата, като дефинирате променлива:

```
int razl;
```

и чрез нея определете разликата между ASCII кодовете на малка и съответната ѝ главна буква:

```
razl = 'a' - 'A';
```

Проследете и анализирайте резултата.

Забележете, че с данните от тип `char` могат да се извършват аритметични операции, тъй като символите имат съответни числови кодове.

Какъв резултат ще получите за:

```
'a' - 'A'
```

и защо?

Задача 1.5.

Условието и структурата на примерната програма са свързани с обработка на големи цели числа.

За пресмятане броя на зърната се използва програмната конструкция **цикъл**, реализирана чрез два вложени оператора `for`.

Според известната легенда за създателя на шахматната игра на първото поле на шахматната дъска се поставя едно житено зърно, на второто – два пъти повече, на третото – отново два пъти повече и т.н.

Следователно броят на зърната е:

```
1 + 2 + 4 + 8 + ... + 2^63
```

За да се упражни работата с вложени цикли, степента на двойката ще бъде пресмятана чрез вътрешен цикъл.

Ако приемем, че в един килограм има приблизително 5000 житени зърна, то в чувал от 50 kg има приблизително:

```
5000 * 50 = 250000 зърна
```

```
/* Лабораторно упражнение 1. Задача 5. */
/* C++ програма за пресмятане броя на чувалите
   със жито според легендата за шахматната дъска. */

/* Вариант с два вложени цикъла for –
   външен с индекс i и вътрешен с индекс j. */

#include <iostream>
#include <iomanip>

int main()
{
    unsigned long long obshto_zyrna = 0;

    for (int i = 1; i <= 64; i++)
    {
        unsigned long long zyrna = 1;

        // Пресмятане на 2^(i - 1)
        for (int j = 1; j < i; j++)
        {
            zyrna = zyrna * 2;
        }

        // Натрупване на общия брой зърна
        obshto_zyrna = obshto_zyrna + zyrna;
    }

    long double ch_jito =
        static_cast<long double>(obshto_zyrna) / 250000.0L;

    std::cout << "Общият брой житени зърна е: "
              << obshto_zyrna << '\n';

    std::cout << std::scientific << std::setprecision(6);

    std::cout << "Приблизителният брой чували със жито е: "
              << ch_jito << '\n';

    return 0;
}
```

Въведете, компилирайте, тествайте и изпълнете програмата.

За изпълнението на програмата не е необходимо въвеждането на входни данни.

Полученият общ брой зърна трябва да бъде:

```
18446744073709551615
```

т.е.

```
2^64 - 1
```

При условие, че един чувал съдържа приблизително 250000 зърна, броят на чувалите е приблизително:

```
7.378698 × 10^13
```

Променете броя на квадратчетата от 64 на 10.

Изпълнете модифицираната програма и сравнете програмно получения резултат с резултата от ръчно пресмятане:

```
1 + 2 + 4 + 8 + ... + 512
```

Каква е получената сума?

Съставете и изпълнете вариант на програмата, като замените двата вложени цикъла `for` със съответни цикли с предусловие:

```
while
```

След това опитайте да реализирате задачата само с един цикъл, като при всяка итерация удвоявате броя на зърната.

Въпроси и задачи за самостоятелна работа

1. На кои основни елементи, като йерархична система, се базира езикът за програмиране C++?

2. Какво е предназначението на заглавните (header) файлове, включвани в C++ програмите?

3. С коя предпроцесорна директива се включват заглавни файлове в C++ програма?

4. Какви са различията при дефинирането и използването на константите и променливите в C++ програма?

5. Кои дефинирани променливи се наричат локални? Каква е тяхната област на видимост?

6. Какво определя типът на данните в езика C++?

7. Кои са основните вградени типове данни, които могат да бъдат използвани в C++?

8. Какво е предназначението на оператора за присвояване и какво е характерно при неговото изпълнение?

9. Какво е предназначението на стандартните потоци:

```
std::cin
std::cout
```

и в кой заглавен файл са декларирани?

10. Какво е предназначението на заглавния файл:

```
<cmath>
```

Посочете поне две математически функции, които могат да бъдат използвани чрез него.

11. Каква е разликата между типовете:

```
float
double
long double
```

Защо при изчисления, изискващи по-голяма точност, обикновено се предпочита `double` пред `float`?

12. Съставете и изпълнете C++ програма за намиране сумата `sumc` от цифрите на трицифрено цяло число, въвеждано от клавиатурата.

Пример:

```
Вход:
527

Резултат:
5 + 2 + 7 = 14
```

Използвайте операциите `/` и `%`.

13. Съставете C++ програма за изчисляване периметъра `P` на правилен многоъгълник по въведени от клавиатурата:

- брой страни – `br`;
- размер на страната – `str`.

Използвайте зависимостта:

```
P = br * str
```

14. Съставете програма за изчисляване сумата от кодовете на латинските букви в зададен символен низ.

Например програмата да въвежда:

```
KST
```

и да извежда кода на всеки символ и тяхната сума.

Използвайте тип:

```
std::string
```

и заглавния файл:

```
<string>
```

15. Обяснете разликата между следните два оператора:

`=`

и

`==`

Дайте пример за използването на всеки от тях.

16. Каква е разликата между:

`&&`

и

`||`

в логически израз?

17. Каква е ролята на:

```
return 0;
```

във функцията `main()`?

18. Каква е разликата между циклите:

```
for
while
do ... while
```

и в какви случаи е удобно да се използва всеки от тях?

19. Какво ще бъде изведено от програмния фрагмент:

```
int a = 7;
int b = 3;

std::cout << a / b << '\n';
std::cout << a % b << '\n';
```

Обяснете получения резултат.

20. Съставете кратка C++ програма, която въвежда две реални числа и извежда:

- тяхната сума;
- разликата им;
- произведението им;
- частното им.

Предвидете проверка дали второто число е различно от нула преди извършване на делението.